From Milliseconds to Microseconds: Optimizing the Static Forward Kinematics of CTCRs for use in Real-Time Systems

Taqi Hamoda

***Abstract*— Efficient computation of the forward kinematics (FK) for concentric tube continuum robots (CTCRs) is essential for real-time control and deployment. However, the current gold-standard model, the static Cosserat rod model, is computationally intensive, limiting its use in high-frequency applications. Despite various optimization efforts, a universal strategy that achieves real-time performance has remained elusive. We tackle this challenge by applying a series of numerical optimizations, including explicit integration techniques, quaternion-based orientation representation, and efficient Jacobian updating schemes. These optimizations reduce function calls by up to 90% without sacrificing accuracy, achieving a runtime of just 120 microseconds for FK computation, enabling real-time control while ensuring universal applicability across various architectures and systems. Additionally, we introduce a novel, computationally accessible proxy for elastic stability, which could enable the detection of snapping behavior in CTCRs.**

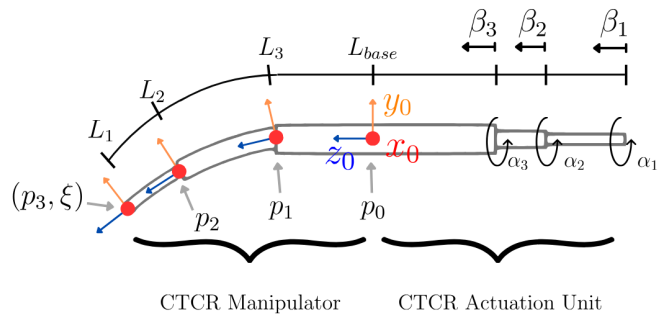

Fig. 1: Diagram of an annotated CTCR consisting of three precurved tubes. Please note that the innermost, middle, and outermost tubes are referenced with the indices 1, 2, and 3 respectively.

I. INTRODUCTION

Concentric tube continuum robots (CTCRs), as shown in Fig. 1, consist of multiple concentric, precurved, super-elastic tubes that can be rotated and translated relative to one another [1], [2]. Due to the complex elastic interactions between the tubes, numerical tools are often needed to capture the highly non-linear behaviour of robots with more than two tubes [3]. Rucker et al. introduced in [4] a highly-accurate physics model based on Cosserat rod theory, formulating the robot's kinematics as a boundary value problem (BVP) with unknown initial conditions. Although this model is the gold-standard for CTCR modeling, there is a high computational cost associated with solving its BVP.

Efforts to optimize the computational cost of solving the Cosserat rod model have been extensive, yet no universal approach has emerged. While some studies offer broad, architecture-agnostic improvements, they often fall short of real-time control, achieving execution times of 40 ms and 57 ms respectively [5], [6]. Other optimizations achieve real-time performance but are limited by specific robot architectures [7] or actuation systems [8]. Despite these advancements, there remains a critical need for an optimization strategy that not only achieves real-time control but is also universally applicable, balancing generalized performance with real-time execution across various systems.

This paper presents a comprehensive investigation of the numerical tools used in the computation of the static forward kinematics (FK) of CTCRs. We introduce a variety of numerical optimization techniques that result in significant runtime reductions with no compromise in accuracy, enabling real-time performance while maintaining universal applicability across model implementations. Additionally, we demonstrate a potential numerical proxy for assessing the robot's elastic stability which opens up new possibilities for stable path planning in CTCR systems.

A. State of the Art

Current state-of-the-art approaches for solving the BVP primarily use explicit Runge-Kutta methods combined with the Newton-Raphson nonlinear solver [4], [5], [9], [10]. This combination is favored due to its robustness and ease of use, particularly in environments like MATLAB [4], [5], [7]. Although it was initially believed that specialized integrators were needed to preserve rotation group structure [3], the error from standard explicit integrators has been shown to be negligible compared to the model's inherent kinematic error [9], [11]. The shooting method is commonly employed in this framework, effectively guiding the solution towards convergence.

II. THE BOUNDARY VALUE PROBLEM

This section discusses the mathematical formulation of the BVP for CTCRs. It introduces the Cosserat rod model, examines the stiffness of the model, and reviews existing methods, setting the foundation for the computational benchmarks explored later.

A. The Cosserat Rod Model

Cosserat rod theory provides a comprehensive mathematical framework for modeling slender, one-dimensional rods by accounting for bend, twist, stretch, and shear to represent all six degrees of freedom at each cross-section [4]. In the context of CTCRs, Kirchhoff rod theory, a special case of Cosserat rod theory that assumes inextensibility

NOMENCLATURE

$\boldsymbol{a}$	Bold typeface for vectors
A	Uppercase typeface for matrices
n	Number of concentric, precurved tubes
s	Point along the robot's arc-length
$(\cdot)^*$	Denotes that the variable is in the undeformed state
$\boldsymbol{r}_i(s)$	The position vector for tube i at point s
$R_i(s)$	The rotation matrix for tube i at point s
$g_i(s)$	The reference frame for tube i at point s
E_i	Young's modulus for tube i
G_i	Shear modulus for tube i
I_i	The second moment area for tube i
J_i	The polar moment of inertia of the ith tube's cross-section
K_i	Stiffness matrix for tube i
$(\cdot)^\vee$	Denotes the conversion from $\mathfrak{so}(3)$ or $\mathfrak{se}(3)$ to $\mathbb{R}^3$ or $\mathbb{R}^6$ respectively
$\widehat{\cdot}$	The inverse function of $(\cdot)^\vee$
$\boldsymbol{u}_i(s)$	Precurvature vector for tube i at point s
θ_i	Axial angle of the ith tube
$\boldsymbol{\xi}$	Rate of change of the reference frame g
$u_{i,z}(s)$	Torsional strains for tube i at point s
α_i	Rotational joint value corresponding to the ith tube
β_i	Translational joint value corresponding to the ith tube
$\boldsymbol{q}$	The robot's joint space vector
L_i	Length of the ith tube
$\boldsymbol{y}$	State vector for the ODE system
$\boldsymbol{f}(s,\boldsymbol{y})$	Derivative of the state vector
$A(s)$	Jacobian matrix of the ODE system at point s
$\lambda_j(s)$	Eigenvalue of $A(s)$
h	Integration step-size
k	Non-linear solver iteration count
$n_{\boldsymbol{f}}(h)$	Denotes the number of calls to f using a step-size of h
e_p	Position error

and shearlessness, is used. These assumptions are generally valid given the long, thin Nitinol tubes used in CTCRs [4]. In addition to the Kirchhoff assumptions, we will further assume that the tubes' precurvature is constant across the arc length and that the tubes are precurved solely around the x-axis as is shown in Fig. 1.

To model a collection of n concentric, precurved tubes, we must assign homogeneous reference frames along the tubes' arc-lengths. By convention, the z-axes of these frames are always tangent to the curve of the tube, resulting in the frame formulation

$$g_i^*(s) = \begin{bmatrix} R_i^*(s) & \boldsymbol{r_i^*}(s) \\ \boldsymbol{0}^T & 1 \end{bmatrix}.$$

The material properties and the tube's geometry of each cross-section can be described by the stiffness matrix K_i and the local precurvature vector $\boldsymbol{u}_i(s)$ respectively:

$$K_i = \begin{bmatrix} E_iI_i & 0 & 0 \\ 0 & E_iI_i & 0 \\ 0 & 0 & G_iJ_i \end{bmatrix}, \quad \boldsymbol{u}_i^*(s) = \left(R_i^{*T}(s)\dot{R}_i^*(s)\right)^\vee.$$

Lastly, by initializing the following state variables as

$$R_{\theta_i} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 \\ \sin\theta_i & \cos\theta_i & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad K = \sum_{i=1}^{n} K_i$$

$$\boldsymbol{u}_i = R_{\theta_i}^T \boldsymbol{u}_1 + \dot{\theta}_i \boldsymbol{e}_3 \qquad \boldsymbol{\xi} = \begin{bmatrix} 0 & 0 & 1 & \boldsymbol{u}_1^T \end{bmatrix}^T,$$

an ODE system can be constructed using the Cosserat rod equations to describe the tubes' shapes along with the elastic interactions between them:

$$\begin{aligned} \dot{g} &= g\widehat{\boldsymbol{\xi}} \\ \dot{\theta}_i &= u_{i,z} - u_{1,z} \\ \begin{bmatrix} \dot{u}_{1,x} \\ \dot{u}_{1,y} \end{bmatrix} &= -K^{-1}\Big(\sum_{i=1}^{n} R_{\theta_i}\big(K_i(\dot{\theta}_i \frac{dR_{\theta_i}^T}{d\theta_i}\boldsymbol{u}_1) \\ &\quad + \widehat{\boldsymbol{u}}_i K_i(\boldsymbol{u}_i - \boldsymbol{u}_i^*)\big) + \widehat{e}_3 R_1^T \boldsymbol{w}_f\Big)\Big|_{x,y} \\ \dot{u}_{i,z} &= \frac{E_iI_i}{G_iJ_i}(u_{i,x}u_{i,y}^* - u_{i,y}u_{i,x}^*). \end{aligned}$$

With constant tube precurvature and zero z-axis curvature, the final boundary condition simplifies to zero torsional strains $u_{i,z}$ at each tube's end. The initial boundary condition at $s = 0$ relates to the robot's joint space and is defined as:

$$\boldsymbol{r}(0) = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T \qquad \theta_1 = \alpha_1 - \beta_1 u_{1,z}$$
$$\theta_i = (\alpha_i - \beta_i u_{i,z}) - \theta_1, \quad i \geq 2.$$

B. Stiffness Analysis of the BVP

The stiffness of an ODE system indicates its conditioning. Stiff ODEs require the use of implicit integrators and/or minuscule step sizes to prevent divergence, while non-stiff ODEs allow the use of explicit integrators with moderate step sizes [12], [13]. Consider the BVP described by the following non-linear ODE system

$$\dot{\boldsymbol{y}} = \boldsymbol{f}(s,\boldsymbol{y}), \quad 0 \leq s \leq b \tag{1}$$

$$A(s) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{y}}(s,\boldsymbol{y}), \tag{2}$$

where $A(s) \in \mathbb{R}^{m\times m}$. A BVP is considered stiff if and only if

$$b|\Re(\lambda_j(s))| \gg 1, \quad 1 \leq j \leq m,$$

[12]. Unlike IVPs, BVP stiffness depends on boundary conditions, allowing it to tolerate error accumulation from positive eigenvalues if the integration interval near the final boundary b is small enough [12].

Following [5], we define our state $\boldsymbol{y}$ as

$$\boldsymbol{y} = [\theta_2, \cdots, \theta_n, u_{1,x}, u_{1,y}, u_{1,z}, u_{2,z}, \cdots, u_{n,z}]^T \tag{3}$$
$$= [y_1, \cdots, y_{n-1}, y_n, y_{n+1}, y_{n+2}, y_{n+3}, \cdots, y_{2n+1}]^T \tag{4}$$

where the first $n-1$ elements determine the position and orientation at s, and the remaining elements describe tube bending and torsion [4]. Deriving an analytical upper bound for the eigenvalues of $A(s)$ is challenging due to the non-linear relationships between the state variables. Instead, we employed a statistical approach, sampling $20,000,000$ configurations uniformly and calculating the maximum $max(|\Re(\lambda_j(s))|)$ across the integration interval.

We use 20 as the upper bound for the system's eigenvalues since $|\Re(\lambda_j(s))| > 20$ has only a 0.001% probability of occurrence, as can be seen in Fig. 2. With a maximum CTCR arc length of 210 mm, we assume

$$0 \leq |\Re(\lambda_j(s))| \leq 20$$
$$0 \leq b|\Re(\lambda_j(s))| \leq 4.2$$

$$A(s) = \begin{bmatrix} 0 & \cdots & 0 & 0 & 0 & -1 & 1 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & 0 & -1 & 0 & \cdots & 1 \\ \frac{\partial f_n}{\partial y_1} & \cdots & \frac{\partial f_n}{\partial y_{n-1}} & \frac{\partial f_n}{\partial y_n} & \frac{\partial f_n}{\partial y_{n+1}} & \frac{\partial f_n}{\partial y_{n+2}} & \frac{\partial f_n}{\partial y_{n+3}} & \cdots & \frac{\partial f_n}{\partial y_{2n+1}} \\ \frac{\partial f_{n+1}}{\partial y_1} & \cdots & \frac{\partial f_{n+1}}{\partial y_{n-1}} & \frac{\partial f_{n+1}}{\partial y_n} & \frac{\partial f_{n+1}}{\partial y_{n+1}} & \frac{\partial f_{n+1}}{\partial y_{n+2}} & \frac{\partial f_{n+1}}{\partial y_{n+3}} & \cdots & \frac{\partial f_{n+1}}{\partial y_{2n+1}} \\ 0 & \cdots & 0 & \frac{\partial f_{n+2}}{\partial y_n} & \frac{\partial f_{n+2}}{\partial y_{n+1}} & 0 & 0 & \cdots & 0 \\ \frac{\partial f_{n+3}}{\partial y_1} & \cdots & 0 & \frac{\partial f_{n+3}}{\partial y_n} & \frac{\partial f_{n+3}}{\partial y_{n+1}} & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \frac{\partial f_{2n+1}}{\partial y_{n-1}} & \frac{\partial f_{2n+1}}{\partial y_n} & \frac{\partial f_{2n+1}}{\partial y_{n+1}} & 0 & 0 & \cdots & 0 \end{bmatrix}$$

The analytical Jacobian of the non-linear ODE presented in Eq. 2.

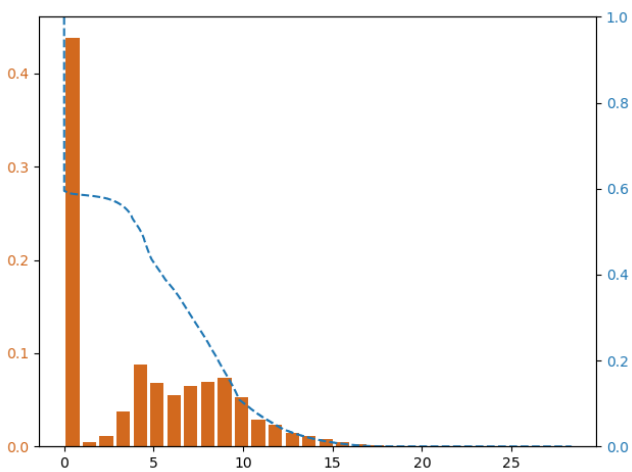

Fig. 2: Histogram of the sampled $max(|\Re(\lambda_j(s))|)$. The orange bars represent the density while the blue line represents the empirical complementary cumulative distribution function (CCDF). A total of $20,000,000$ configurations were sampled.

where $b = 0.21$, indicating that the BVP is non-stiff. While the aforementioned, approximated eigenvalue upper-bound is specific to the CTCR parameters used, the widespread success of using the shooting method with the explicit classical Runge-Kutta integrators [10] suggests that the BVP is generally non-stiff.

III. METHODS

In this section, we explore the numerical methods applied to optimize the static FK of CTCRs. The focus is on numerical integration techniques, orientation representations, and solver strategies, all aimed at improving computational efficiency.

A. Numerical Integration

An $\mathcal{S}$-stage Runge-Kutta (RK) method uses an interpolating polynomial of degree $\mathcal{S}$

$$\boldsymbol{Y}_i = \boldsymbol{y}_{n-1} + h \sum_{j=1}^{i} a_{ij} \boldsymbol{f}(s_{n-1} + c_j h, \boldsymbol{Y}_j)$$

$$\boldsymbol{y}_n = \boldsymbol{y}_{n-1} + h \sum_{i=1}^{\mathcal{S}} b_i \boldsymbol{f}(s_{n-1} + c_i h, \boldsymbol{Y}_i)$$

requiring $\mathcal{S}$ function evaluations per step [12]. In contrast Adams-Bashforth (AB) integrators extrapolate the solution using previous integration steps, requiring only one new function evaluation per step resulting in a comparable performance to Forward Euler (FE) [12]. Nonetheless, the stability region decreases with higher-degree extrapolation, limiting AB methods to problems with low stiffness [12], [13]. The extrapolating polynomial for an $\mathcal{S}$-stage AB method is constructed as follows

$$\boldsymbol{y}_n = \boldsymbol{y}_{n-1} + h \sum_{j=1}^{\mathcal{S}} \gamma_j \boldsymbol{f}(s_{n-j}, \boldsymbol{y}_{n-j}),$$

where γ_j is the coefficient at each interpolating point.

TABLE I: Coefficients of AB methods up to order 5 [12].

$\mathcal{S}$	$j \rightarrow$	1	2	3	4	5
1	γ_j	1				
2	$2\gamma_j$	3	-1			
3	$12\gamma_j$	23	-16	5		
4	$24\gamma_j$	55	-59	37	-9	
5	$720\gamma_j$	1901	-2774	2616	-1274	251

B. Integrating Orientation

A quaternion is a hyper-complex number represented as

$$\boldsymbol{\eta} = a + b\boldsymbol{i} + c\boldsymbol{j} + d\boldsymbol{k},$$
$$\boldsymbol{i}^2 = \boldsymbol{j}^2 = \boldsymbol{k}^2 = \boldsymbol{ijk} = -1,$$

with real coefficients a, b, c, d for the basis vectors $\mathbf{1}, \boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}$ [14]. Quaternions offer a minimal yet complete 3D rotation description, making them a computationally efficient representation of $SO(3)$ [14]. Initially, it was believed that

integrating non-unit quaternions would degrade the $SO(3)$ structure, requiring the enforcement of unit length as an additional algebraic constraint, which was computationally intensive [15]. However, [15] showed that non-unit quaternions can be integrated without constraints using the conventional quaternion derivative equation

$$\dot{\boldsymbol{\eta}} = \frac{1}{2} W^T \boldsymbol{\omega}, \qquad W = \begin{bmatrix} -b & a & d & c \\ -c & -d & a & b \\ -d & c & -b & a \end{bmatrix}$$

where $\omega \in \mathbb{R}^3$ is the body-frame's angular velocity vector, significantly improving accuracy over to Euler angles and rotation matrices [15], [16].

C. Non-Linear Solvers

Let $\boldsymbol{z} : \mathbb{R}^m \to \mathbb{R}^m$ be a differentiable function with $D(\boldsymbol{x})$ as its Jacobian matrix. Iteratively solving $D(\boldsymbol{x})\boldsymbol{d} = -\boldsymbol{z}(\boldsymbol{x})$ using the Newton-Raphson method converges towards a local minimum, corresponding to a root of $\boldsymbol{z}$ [13]. Newton-Raphson achieves quadratic convergence, doubling the number of correct digits with each iteration [13]. However, convergence is not guaranteed if D is singular, $\boldsymbol{z}$ is poorly conditioned, or no root exists. Without a closed-form expression, computing D can be costly since approximating D via finite differences requires at least $m+1$ evaluations of $\boldsymbol{z}$ per iteration, and solving the linear system involves $\mathcal{O}(m^3)$ arithmetic operations, adding significant computational overhead [13].

Algorithm 1 Newton-Raphson's Method

1: $\boldsymbol{x_0} \leftarrow$ initial guess
2: **for** $k = 1, 2, 3, \ldots$ **do**
3: Solve $D(\boldsymbol{x}_{k-1})\boldsymbol{d} = -\boldsymbol{z}(\boldsymbol{x}_{k-1})$ for $\boldsymbol{d}$
4: $\boldsymbol{x}_k \leftarrow \boldsymbol{x}_{k-1} + \boldsymbol{d}$

In one-dimensional problems, the secant update offers a computationally inexpensive alternative to finite-differences for derivative approximations, especially when $\boldsymbol{z}(x)$ is well-conditioned but costly to compute. Despite the secant method's super-linear convergence rate of $(1+\sqrt{5})/2 \approx 1.618$, its lower cost per iteration often outweighs the slightly higher number of iterations needed [13]. In [17], Broyden extended the secant method to multiple dimensions by directly updating the Jacobian matrix using rank-one updates

$$D_k = D_{k-1} + \frac{\Delta \boldsymbol{z}_k - D_{k-1}\Delta \boldsymbol{x}_k}{\|\Delta \boldsymbol{x}_k\|^2} \Delta \boldsymbol{x}_k^T. \tag{5}$$

He also noted that the Sherman–Morrison formula (Eq. 6) can be used to directly update the inverse Jacobian matrix D^{-1}

$$(U + \boldsymbol{u}\boldsymbol{v}^T)^{-1} = U^{-1} - \frac{U^{-1}\boldsymbol{u}\boldsymbol{v}^T U^{-1}}{\boldsymbol{1} + \boldsymbol{v}^T U^{-1}\boldsymbol{u}} \tag{6}$$

where, for some vectors $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, $\boldsymbol{u}\boldsymbol{v}^T$ is the outer product [17]. Applying the Sherman-Morrison formula to Broyden's multi-dimensional secant update (Eq. 5) results in

$$D_k^{-1} = D_{k-1}^{-1} - \frac{\boldsymbol{d}_k \left(-\boldsymbol{d}_{k-1}^T D_{k-1}^{-1}\right)^T}{-\boldsymbol{d}_{k-1}^T(\boldsymbol{d}_k - \boldsymbol{d}_{k-1})} \tag{7}$$

where $\boldsymbol{d}_k = D_{k-1}^{-1}\boldsymbol{z}_k$ is the gradient at iteration k [18]. Please note that the fraction notation implies element-wise or Hadamard division [17].

The availability of the inverse Jacobian allows for direct gradient calculation, with each secant update requiring only $\mathcal{O}(m^2)$ operations per iteration [13]. Moreover, employing the secant update method reduces the number of function calls to $\boldsymbol{z}$ from $m+1$ calls to a single function call per iteration, significantly lowering the computational cost to a level comparable to having a closed-form of D^{-1}.

D. Inverse Jacobian Initialization

Multi-dimensional secant updates typically begin by initializing D^{-1}, often via finite-differences, which can be costly if the iteration count is low. Alternatively, using the identity matrix for D^{-1} might work for well-conditioned problems but may require a higher iteration count to correct the poor initial estimate [13]. In this context, varying the initial step size h_{initial} used in the Jacobian construction can provide control over the approximation level present, with larger h_{initial} resulting in rougher estimates. To minimize total function calls while solving the BVP, the optimal h_{initial} can be determined using

$$h_{\text{optimal}} = \underset{h_{\text{initial}}}{\operatorname{argmin}} \left(n_{\boldsymbol{f}}(h) + n_{\boldsymbol{f},D}(h_{\text{initial}})\right) \tag{8}$$

where $n_{\boldsymbol{f},D}(h_{\text{initial}})$ is the number of function calls used to approximate D^{-1}.

IV. EVALUATION

This section evaluates the performance and accuracy of the CTCR model. It presents the calibration of the CTCR model, followed by a discussion of the sampling method used and the specifics of the model's implementation.

A. Model Implementation

The Cosserat rod static model, outlined in [4], was implemented in C++ and compiled with the g++ compiler. The `-O3` compiler flag was used for the maximum IEEE compliant runtime speed optimization and the `-march=native` flag was used to optimize for the specific processor architecture available. The `_Float64` data type was preferred over `double` for enhanced floating-point processing. Time measurements for solving the forward kinematics were made using C++'s `chrono` library, which offers nanosecond precision. Computations were performed on a 64-bit LINUX system with a Ryzen-5 3500-U 2.1 GHz 4-core, 8-thread processor.

A constant step-size of 1 mm is used in all integrators with zero initial forces as the default guess for all solvers. The Ralston's coefficients for RK integrators were used since they

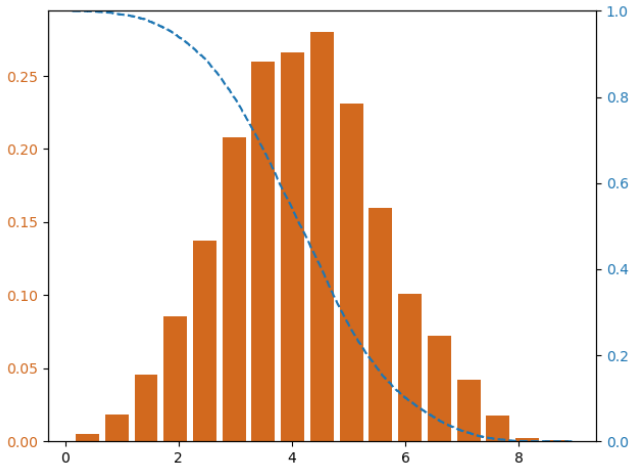

Fig. 3: Histogram of the tip errors $e_p(L_3)$ after calibration. Orange bars indicate the density while the blue line represents the empirical CCDF. Calibration on $10,000$ random data points resulted in a mean of 4.172 ± 1.394 mm and a median of 4.155 mm.

provide the lowest possible truncation error [19]. Ground-truth values were calculated using the Newton-Raphson non-linear solver and the fourth-order RK integrator with a $10\,\mu$m step size, using rotation matrices for orientation. For eigenvalue data collection (Fig. 2) and parameter calibration (Fig. 3), the model used Broyden's update method with the second-order AB integrator.

Each model type, defined by a unique combination of numerical methods, was evaluated on $1,000,000$ random samples, with the averaged results shown in Table IV. To disentangle the β_i values and provide a truly uniform sampling of the workspace, the transformation matrix

$$M_{\mathcal{B}} = \begin{bmatrix} -L_1 & 0 & 0 \\ -L_1 & L_1 - L_2 & 0 \\ -L_1 & L_1 - L_2 & L_2 - L_3 \end{bmatrix}$$

can be used, providing an efficient and bias-free sampling technique [20]:

$$\begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} M_{\mathcal{B}} & M_{\mathcal{B}} \cdot \mathbf{1}_{3\times 1} \end{bmatrix} \begin{bmatrix} \mathcal{U}[-1;1] \\ \mathcal{U}[-1;1] \\ \mathcal{U}[-1;1] \\ 1 \end{bmatrix}.$$

B. Model Calibration

To evaluate the model's performance and ensure correctness, the CTCR model used for data collection was based on the robot parameters used by Grassmann et al. [21]. The authors used an AURORA v2 electromagnetic tracking system to collect a dataset of $100,000$ robot configurations, including position vectors and quaternions for the distal ends, L_{base}, L_3, L_2, and L_1 in the tracking system's reference frame. The dataset is available under an open license. The robot's parameters (Table II) were calibrated by solving a non-linear least squares problem using $10,000$ randomly selected data points. An offset vector $\boldsymbol{\delta_p}$ was applied to the distal end positions to reduce sensor placement uncertainty, and these adjusted positions were then transformed to the robot's base frame. The optimization function was specified as

$$\boldsymbol{P} = \begin{bmatrix} \boldsymbol{\delta}^T_{p,L_{\text{base}}} & \boldsymbol{\delta}^T_{p,L_3} & \boldsymbol{\delta}^T_{p,L_2} & \boldsymbol{\delta}^T_{p,L_1} \end{bmatrix}^T$$
$$\boldsymbol{P}_{\text{optimal}} = \underset{\boldsymbol{P}}{\operatorname{argmin}} \left(e_p(L_1) + e_p(L_2) + e_p(L_3) \right)$$

where $e_p = \|\boldsymbol{r}_{\text{model}}(s) - \boldsymbol{r}_{\text{data}}(s)\|_2$. Calibration was performed using the Trust Region Reflective (TRF) algorithm via SCIPY's `optimize.least_squares` function.

TABLE II: Tube Parameters of the CTCR from [21].

Parameter			Tubes[1]		
Term	Symbol	Unit	inner	middle	outer
Length, overall	L	mm	210	165	110
Length, straight	L_{s}	mm	41	100	100
Curvature[2]	κ_x	m^{-1}	28	12.4	4.37
Diameter, outer[2]	d_{outer}	mm	0.5	0.9	1.5
Diameter, inner[2]	d_{inner}	mm	0.4	0.7	1.2
Young's Modulus	E	GPa	50	50	50
Poisson's ratio	ν	1	0.3	0.3	0.3

[1] The inner, middle, and outer tubes are referenced to 1, 2, and 3, respectively.
[2] The tube diameters and curvature are taken from the manufacturer's data sheet.

The average tip position error was 4.172 mm which is 1.99% of the total arc length. Approximately 92.7% of the tip errors were below 6.3 mm, or 3% of the total arc length, which is consistent with the literature [10], [9]. Further fine-tuning of the position error can be achieved by calibrating the stiffness matrix values, as discussed in [4].

TABLE III: Values of Calibrated Parameters

Tube End	δx in mm	δy in mm	δz in mm
L_{base}	2.179	11.264	1.006
L_3	1.844	−0.184	0.504
L_2	−0.523	−4.690	−1.321
L_1	−3.501	−6.389	−0.177

C. Computational Benchmark

In this paper, the primary benchmark for measuring computational complexity will be a combination of the number of function calls to $\boldsymbol{f}(s, \boldsymbol{y})$ and computational runtime. While other methods, such as asymptotic complexity analysis and runtime measurements alone, have their merits, they often fall short in capturing the intricate interactions involved in optimization, such as constant runtime improvements due to quaternion representation. By focusing on function call count and runtime, we provide a more accurate and accessible metric that directly reflects the optimization level achieved.

D. Results

Across all model types, the iteration count remained relatively low with 99.8% of all sampled configurations

TABLE IV: Average Performance of Different Numerical Models

	Newton – Rotation Matrix				Broyden – Rotation Matrix				Broyden – Quaternion			
Integrator	e_p in mm	n_f	k	t in μs	e_p in mm	n_f	k	t in μs	e_p in mm	n_f	k	t in μs
FE	2.302	1790	4.36	**305**	2.302	963	**5.12**	**164**	2.229	**963**	5.13	**155**
AB 2	2.041	**1787**	**4.35**	327	2.039	**962**	**5.12**	184	2.037	**963**	5.13	172
AB 3	2.034	1788	**4.35**	348	2.034	963	5.13	191	2.028	**963**	5.13	182
AB 4	2.035	1790	4.36	374	2.035	963	**5.12**	204	2.029	**963**	5.13	200
AB 5	2.035	1789	**4.35**	405	2.035	964	5.13	217	2.028	**963**	5.13	212
RK 2	2.027	3575	**4.35**	537	2.027	1924	5.13	292	**2.025**	1925	**5.12**	288
RK 3	2.025	5366	4.36	785	**2.025**	2889	5.13	426	**2.025**	2887	5.13	413
RK 4	**2.024**	7151	**4.35**	1032	**2.025**	3851	5.13	558	**2.025**	3847	**5.12**	541

converging successfully. The position error was virtually identical between Newton and Broyden non-linear solvers; nonetheless, the Broyden update method reduced function calls by about 54%. Although using RK integrators resulted in a slightly more accurate solution when compared to AB methods, the difference in the position error was $16\,\mu\mathrm{m}$ on average making it negligible for practical purposes. With a 5% reduction in runtime and no change in function calls, the quaternion-based model offers improved stability during integration, highlighting the importance of constant runtime improvements for enhancing overall efficiency.

Using the second-order AB integrator instead of the fourth-order RK method leads to a 75% reduction in function calls with only a 0.8% increase in position error. Although FE resulted in the highest position error across all integrators, the difference in position error between it an fourth-order RK is only $0.278\,\mathrm{mm}$ or 0.1% of the CTCR's arc-length, making it sufficient for practical use unless high accuracy is essential. To determine the appropriate level of estimation for initializing the inverse Jacobian, a variety of h_{initial} were examined using the Broyden model with FE and quaternion integration. For this specific CTCR parameter set, the optimal h_{initial} appears to be around $7\,\mathrm{mm}$, achieving a 25% reduction in the number of function calls with no effect on the model's accuracy (Table V).

TABLE V: Performance of Various h_{initial} Values.

h_{initial} in mm	e_p in mm	n_f	k	t in μs
3	**2.228**	753	**5.79**	124
4	**2.228**	735	5.95	122
5	2.230	729	6.09	121
6	2.229	727	6.20	121
7	**2.228**	**726**	6.30	**120**
8	**2.228**	729	6.39	121

V. DISCUSSION

Given the small number of solver iterations, constant runtime improvements play a crucial role for further runtime gains as can be seen by the improvements achieved by the quaternion model. Although quaternion integration didn't affect the accuracy of RK methods significantly, a stabilizing effect can be seen on AB methods, specifically on the higher-order methods. Furthermore, although the decreased stability region of AB methods was thought to be a concern, the constant number of solver iterations k across the different integration methods implies that the truncation error introduced by the integrator is minor relative to the kinematic error of the model. Nonetheless, AB methods experience a proportional increase in runtime in relation to the order of the method due to the additional computations required for the extrapolating polynomial.

Combining all the optimizations discussed above has reduced the function call count by 90% achieving an average static FK model runtime of $120\,\mu\mathrm{s}$ ($8.3\,\mathrm{kHz}$), qualifying its use for high frequency, real-time systems. Given that our proposed approach focuses on numerical enhancements instead of analytical methods, it can provide universal runtime enhancements regardless of the robot's architecture or actuation system used. Additionally, analytical approaches may experience further optimization improvements by utilizing our approach alongside the optimized model due to the non-conflicting natures of both methods [5], [6]. It is worth noting that the proposed approach may provide limited optimization gains for models that do not rely on standard explicit integrators and/or numerical non-linear solvers, although such models are uncommon in the literature [10].

A. Numerical Proxy for Elastic Stability

Snaps or bifurcations in a CTCR system occur due to built-up torsional energy, causing a sudden configuration change as the robot releases the stored tension [23]. This snapping happens when the robot suddenly transitions from a local minimum energy state to a global minima, forcing a snap to occur [23]. Elastic instability in a CTCR increases as the relative axial angle – the absolute difference between the actuator's rotations – approaches π, where instability is maximized [23], [24]. After a bifurcation, the system reaches a stable equilibrium state, which typically occurs when the robot's tubes are aligned or anti-aligned [22].

Although the static Cosserat Rod equations do not model dynamic energy interactions, they do capture the torsional energy present within the tubes. As such, the system's elastic stability can be indirectly inferred from the ODE system's conditioning, with configurations at the equilibrium points resulting in a well-conditioned system. To test our hypothesis, we sampled 20,000,000 random configurations and generated mesh plots of the non-linear solver's iteration count k and the BVP's stiffness value $|\Re(\lambda_j(s))|$ relative to the axial angles. These values indicate the system's

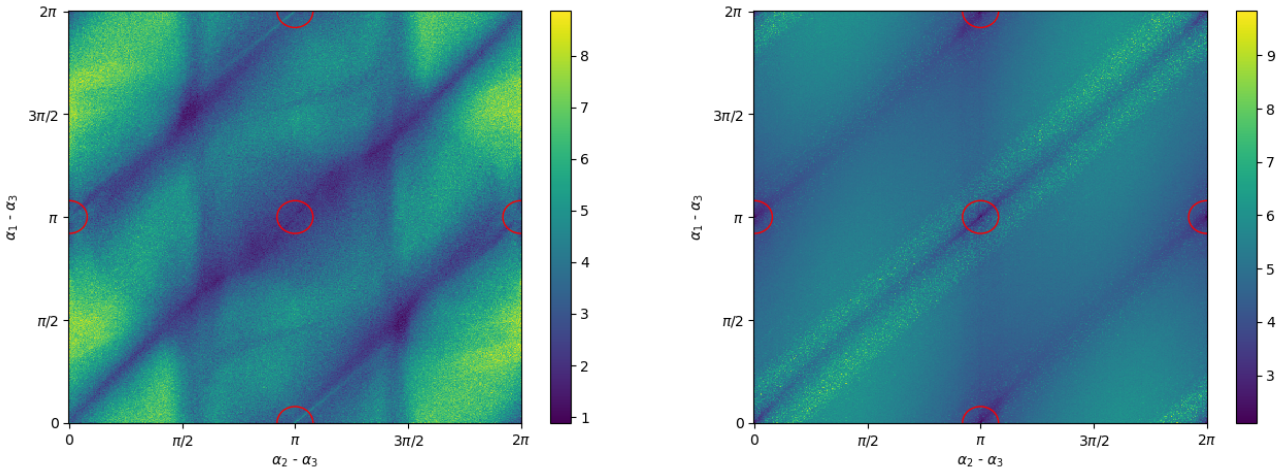

Fig. 4: Mesh grids of $|\Re(\lambda_j(s))|$ (left) and solver iterations k (right) with respect to the relative axial angles. $20,000,000$ random configurations were sampled. The stability equilibria points are $(0,\pi)$, $(\pi,0)$, (π,π), $(\pi,2\pi)$, and $(2\pi,\pi)$ [22].

conditioning, as they are influenced by the conditioning of A [13].

As shown in Fig. 4, areas with low stiffness overlap with equilibrium points. Furthermore, the non-linear solver converges the quickest around regions of equilibria, with clusters of minimal iteration counts centered exactly at the equilibrium points. Regions surrounding points with no changes in the relative axial angles, such as $(0,0)$ and $(2\pi,2\pi)$, also show fast convergence due to the initial guess of zero torsional forces at the base coinciding with the correct solution. Although elastic stability might affect stiffness, other factors seem to have a stronger influence. This is evident from the clustering of eigenvalues at locations other than the equilibrium points.

B. Future Work

Although the Jacobian matrix A was introduced for the ODE system governing the BVP, an analytical upper bound for it has yet to be derived. Such an upper bound would provide deeper insights into how the robot's parameters affect stiffness and identify which elements most significantly influence the system's conditioning. It could also clarify the clustering behavior of the $|\Re(\lambda_j(s))|$ values and explain why they differ from solver convergence. Furthermore, additional runtime improvements may be possible if a relationship between the optimal torsional values at the base and the eigenvalues of A were to be discovered. A thorough examination of A would reveal whether such a relationship exists and how it could be exploited.

Further research is necessary to determine whether the convergence count k consistently serves as a reliable indicator of relative elastic stability. This investigation should explore the behavior of this numerical proxy under various scenarios, including extreme use cases, and demonstrate its practical application. The numerical proxy should also be integrated into a path planner, with the algorithm's performance evaluated in detail. If the numerical proxy proves reliable, it could be leveraged for real-time path planning and the avoidance of snapping behavior [22].

VI. CONCLUSIONS

This investigation covers explicit integration techniques, quaternion-based orientation representation, and efficient Jacobian updating schemes, resulting in significant reductions in the computational runtime of the forward kinematics of the Cosserat Rod model. By combining the proposed optimizations, we demonstrate a reduction of up to 90% in function calls with no loss in accuracy. Our optimized kinematic model achieves an average runtime of just $120\,\mu$s, making it suitable for high-frequency, real-time control of CTCR systems while also ensuring broad applicability across various architectures and systems. Furthermore, we introduced a novel, computationally accessible proxy for elastic stability, which holds promise for detecting snapping behavior in CTCRs.

REFERENCES

[1] R. J. Webster, A. M. Okamura, and N. J. Cowan, "Toward active cannulas: Miniature snake-like surgical robots," in 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2006, pp. 2857–2863.

[2] P. Sears and P. Dupont, "A steerable needle technology using curved concentric tubes," in 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2006, pp. 2850–2856.

[3] P. E. Dupont, J. Lock, B. Itkowitz, and E. Butler, "Design and control of concentric-tube robots," IEEE Transactions on Robotics, vol. 26, no. 2, pp. 209–225, 2010.

[4] D. C. Rucker, B. A. Jones, and R. J. Webster III, "A geometrically exact model for externally loaded concentric-tube continuum robots," IEEE Transactions on Robotics, vol. 26, no. 5, pp. 769–780, 2010.

[5] D. C. Rucker and R. J. Webster, "Computing jacobians and compliance matrices for externally loaded continuum robots," in 2011 IEEE International Conference on Robotics and Automation, 2011, pp. 945–950.

[6] A. L. Orekhov and N. Simaan, "Solving cosserat rod models via collocation and the magnus expansion," in 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020, pp. 8653–8660.

[7] J. Till, C. E. Bryson, S. Chung, A. Orekhov, and D. C. Rucker, "Efficient computation of multiple coupled cosserat rod models for real-time simulation and control of parallel continuum manipulators," in 2015 IEEE International Conference on Robotics and Automation, 2015, pp. 5067–5074.

[8] R. Xu and R. V. Patel, "A fast torsionally compliant kinematic model of concentric-tube robots," in 2012 Annual International Conference of the IEEE Engineering in Medicine and Biology Society, 2012, pp. 904–907.

[9] H. B. Gilbert, D. C. Rucker, and R. J. Webster III, Concentric Tube Robots: The State of the Art and Future Directions. Springer International Publishing, 2016, pp. 253–269.

[10] J. Burgner-Kahrs, D. C. Rucker, and H. Choset, "Continuum robots for medical applications: A survey," IEEE Transactions on Robotics, vol. 31, no. 6, pp. 1261–1280, 2015.

[11] L. Dieci, R. D. Russell, and E. S. Van Vleck, "Unitary integrators and applications to continuous orthonormalization techniques," SIAM Journal on Numerical Analysis, vol. 31, no. 1, pp. 261–281, 1994.

[12] U. M. Ascher and L. R. Petzold, Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations. SIAM, 1998.

[13] M. T. Heath, Scientific Computing: An Introductory Survey, revised second edition. SIAM, 2018.

[14] J. Stuelpnagel, "On the parametrization of the three-dimensional rotation group," SIAM Review, vol. 6, no. 4, pp. 422–430, 1964.

[15] C. Rucker, "Integrating rotations using nonunit quaternions," IEEE Robotics and Automation Letters, vol. 3, no. 4, pp. 2979–2986, 2018.

[16] R. M. Grassmann and J. Burgner-Kahrs, "On the merits of joint space and orientation representations in learning the forward kinematics in se(3)," in 2019 Robotics: Science and Systems XV, 2019.

[17] C. G. Broyden, "A class of methods for solving nonlinear simultaneous equations," Mathematics of Computation, vol. 19, no. 92, pp. 577–593, 1965.

[18] M. Lavrov, "Chapter 3, lecture 6: Broyden's method," 2019, lecture notes for Math 484: Nonlinear Programming, University of Illinois at Urbana-Champaign, Spring Semester.

[19] A. Ralston, "Runge-kutta methods with minimum error bounds," Mathematics of Computation, vol. 16, no. 80, pp. 431–437, 1962.

[20] R. M. Grassmann, A. Senyk, and J. Burgner-Kahrs, "On the disentanglement of tube inequalities in concentric tube continuum robots," in 2024 IEEE International Conference on Robotics and Automation, 2024.

[21] R. M. Grassmann, R. Z. Chen, N. Liang, and J. Burgner-Kahrs, "A dataset and benchmark for learning the kinematics of concentric tube continuum robots," in 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2022, pp. 9550–9557.

[22] H. B. Gilbert, R. J. Hendrick, and R. J. Webster III, "Elastic stability of concentric tube robots: A stability measure and design test," IEEE Transactions on Robotics, vol. 32, no. 1, pp. 20–35, 2016.

[23] R. J. Webster, J. M. Romano, and N. J. Cowan, "Mechanics of precurved-tube continuum robots," IEEE Transactions on Robotics, vol. 25, no. 1, pp. 67–78, 2009.

[24] C. Bergeles, A. H. Gosline, N. V. Vasilyev, P. J. Codd, P. J. del Nido, and P. E. Dupont, "Concentric tube robot design and optimization based on task and anatomical constraints," IEEE Transactions on Robotics, vol. 31, no. 1, pp. 67–84, 2015.